Auditing of NPM packages

Auditing npm packages in a React.js application helps identify **security vulnerabilities, outdated dependencies, incompatible packages, licensing risks, and unnecessary dependencies**.

Five levels of Auditing For react applications:

1. **Dependency inventory** — Identify all installed npm packages.
2. **Security audit** — Detect known vulnerabilities in direct and transitive dependencies.
3. **Dependency maintenance** — Identify outdated and unsupported packages.
4. **Compatibility testing** — Verify that package updates do not break the React application.
5. **CI/CD integration** — Automate auditing in GitHub Actions or another build pipeline.

Steps for React applications created with Vite, Create React App, or similar tooling.

Step 1: Open the React Project

Open a terminal in the React project directory.

For example, on Windows:

```
cd c:\advreactapp
```

Verify that Node.js and npm are installed:

```
node --version
npm --version
```

Check the project structure:

```
react-app/
├── node_modules/
├── public/
├── src/
├── package.json
├── package-lock.json
└── vite.config.js
```

Note: The package.json file declares the project's dependencies, while package-lock.json records the resolved dependency tree and versions used by npm.

For reproducible dependency installation, commit both files to source control.

Step 2: Identify All npm Dependencies

Run: npm ls --depth=0

This lists the top-level installed dependencies.

Example:

```
C:\advreactapp>npm ls --depth=0

advreactapp@0.1.0 C:\advreactapp
+-- @testing-library/dom@10.4.2
+-- @testing-library/jest-dom@6.9.1
+-- @testing-library/react@16.3.3
+-- @testing-library/user-event@13.5.0
+-- react-dom@19.3.1
+-- react-router-dom@7.18.5
+-- react-scripts@5.0.1
```

+-- react@19.3.1
`-- web-vitals@2.1.4

To inspect production dependencies only:

```
npm ls --omit=dev
```

Check :

- Are all expected packages installed?
- Are there missing or invalid dependencies?
- Are multiple versions of the same package installed?
- Are unnecessary packages included?
- Are development-only tools incorrectly included in production dependencies?

A package can introduce risk even if your application never imports it directly, because another dependency may install it.

Step 3: Run the npm Security Audit

Run: `npm audit`

npm compares the project's dependency tree against known vulnerability advisories and reports findings, their severity, affected dependency paths, and available remediation options.

Severity levels

Severity	Risk	Recommended action
Critical	Extremely serious vulnerability	Investigate and remediate immediately
High	Serious security vulnerability	Prioritize an urgent fix
Moderate	Meaningful security risk	Assess impact and schedule remediation promptly
Low	Lower-severity vulnerability	Review and address appropriately
No known vulnerabilities	No matching known advisories found	Continue the remaining audit checks

Example audit output

found 3 vulnerabilities
2 moderate
1 high

To address issues that do not require attention,

```
npm audit fix
```

This is an illustrative example.

Generate a detailed report

For a more detailed report:

```
npm audit --json
```

To save the report to a file in a Windows terminal:

```
npm audit --json > npm-audit-report.json
```

Audit production dependencies separately

```
npm audit --omit=dev
```

This helps you focus on the dependencies included in a production installation. However, development dependencies also matter because they can introduce risks into your build and development environment.

You should generally run both the full audit and the production-focused audit.

Step 4: Fix Vulnerable Packages

After reviewing the audit report, try the standard remediation command:

```
npm audit fix       or npm audit fix --force
```

This attempts to install compatible dependency updates that resolve reported vulnerabilities.

Before running it if you are using GIT:

- Commit your current changes to Git.
- Ensure the working tree is clean or back up your changes.
- Review the proposed changes to package.json and package-lock.json.
- Run the application tests after remediation.

If vulnerabilities remain

Run: npm audit

Identify the affected package and inspect its dependency path.

For example:

```
your-react-app
└── some-ui-library
    └── vulnerable-package
```

Here, vulnerable-package is a transitive dependency. Updating some-ui-library may resolve the issue even if you never installed vulnerable-package directly.

Check why the package is installed:

```
npm explain vulnerable-package
```

Replace vulnerable-package with the actual package name from your audit report.

When a major version upgrade is required

If the audit report recommends upgrading a library to a new major version, inspect the release notes and breaking changes before updating it.

Avoid using this command as your first response:

```
npm audit fix --force
```

The --force option can permit changes outside the existing version ranges, including potentially breaking major-version upgrades. Use it

only after reviewing the changes and planning the required compatibility.

Step 5: Check Outdated npm Packages

Security auditing does not tell you everything about package maintenance.

Run: npm outdated

Understand the output

Column	Meaning
Package	npm package name
Current	Installed version
Wanted	Latest version satisfying the version range in package.json
Latest	Version identified by the registry's latest tag
Location	Where the package is installed

How to update packages?

For compatible updates:

```
npm update
```

For a specific package:

```
npm update axios
```

Best practice: Do not update every package blindly. Review the changelog, compatibility requirements, security implications, and automated test results.

Step 6: Check for Invalid or Conflicting Dependencies

Run: npm ls

Look for messages such as:

invalid
missing
extraneous

These indicate potential dependency consistency problems.

To investigate a particular package:

npm explain react

To check the complete dependency tree for duplicate or conflicting versions, inspect the output of:

npm ls --all

What should you investigate?

- Multiple versions of React or React DOM.
- Packages with incompatible React peer dependencies.
- Missing dependencies.
- Dependencies installed but not declared correctly.
- Packages that are no longer used.
- Old packages that have been replaced by maintained alternatives.

For React applications, pay particular attention to compatibility between react, react-dom, routing libraries, UI component libraries, testing tools, and build tooling.

Step 7: Audit Package Quality and Supply-Chain Risks

A package may have no reported vulnerability but still present a maintenance or supply-chain risk.

For important dependencies, review the following:

Audit area	What to check
Maintenance	Are releases and issues handled regularly?
Compatibility	Does the package support your Node.js and React versions?
Repository	Is the source repository identifiable and credible?
Release history	Are releases consistent with the project's maintenance history?
Dependencies	Does the package introduce an unusually large dependency tree?
Install scripts	Does installation execute scripts that deserve security review?
License	Is the license compatible with your organization's requirements?
Package integrity	Can the package's origin and integrity be verified?

Step 7: Test the React Application After Fixing Vulnerabilities

After updating packages, make sure the application still works.

Run linting

If ESLint is configured in your project:

```
npm run lint
```

The exact script depends on your package.json.

Run unit tests

```
npm test
```

Some React projects use Vitest or Jest and may define different script names. Check package.json before executing this command.

Build the application

For a typical Vite application: npm run build

Final validation checklist

- [ ] No unresolved critical or high-severity vulnerabilities without an approved exception.
- [ ] Moderate and low findings reviewed.
- [ ] Dependencies are compatible with the application.
- [ ] React components render correctly.
- [ ] Routing and navigation work.
- [ ] API calls and authentication work.
- [ ] Unit tests pass.

- [ ] Production build succeeds.
- [ ] The lockfile changes are reviewed.
- [ ] The application has been tested in an appropriate environment.

Step 8: Automate npm Auditing in GitHub Actions

For a professional React project, run dependency audits automatically during continuous integration.

Create the following file:

```
.github/
└── workflows/
    └── npm-audit.yml
```

Add this workflow:

```
name: React npm Security Audit

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  schedule:
    - cron: "30 2 * * 1"

permissions:
  contents: read

jobs:
```

```
audit:
  runs-on: ubuntu-latest

  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Set up Node.js
      uses: actions/setup-node@v4
      with:
        node-version: 22
        cache: npm

    - name: Install dependencies
      run: npm ci

    - name: Audit npm dependencies
      run: npm audit --audit-level=high

    - name: Build React application
      run: npm run build
```